BIOST 2155: Assignment 1

Solutions

Released: 2025-10-17

Instructions

- This file serves as a template for your assignment.

```
pacman::p_load(formatR)
knitr::opts_chunk$set(echo = TRUE, eval = TRUE,
                      fig.height = 4, fig.width = 6, fig.align = "center",
                      warning = FALSE,
                      message=FALSE, tidy.opts = list(width.cutoff = 60), tidy = TRUE,
                      class.output  = "shadebox",
                      class.message = "shadebox",
                      class.warning = "shadebox",
                      class.error   = "shadebox")
```

- Use comments to annotate and explain your code whenever possible.
- This assignment uses one data set: `Boston Housing` and is saved as `assign1_regData.csv`

Theory (10 points)

T1. Mathematical basis of variance-bias tradeoff (10 points)

Assume $Y = f(X) + \epsilon$, where $E(\epsilon) = 0, \text{Var}(\epsilon) = \sigma^2$ and ϵ and $f(X)$ are uncorrelated. Prove the key formula for bias-variance tradeoff:

$$\text{MSE} = (\text{bias})^2 + \text{variance} + \text{irreducible error}$$

where $\text{MSE} = E\left[(y - \hat{f}(x))^2\right]$, bias $= E[\hat{f}(x)] - f(x)$, variance $= \text{Var}[\hat{f}(x)]$ and irreducible error $= \sigma^2$.

Step 1: Expand the MSE: Substitute $Y = f(X) + \epsilon$ into the MSE formula and expand it.

$$\begin{aligned}
\text{MSE} &= E\left[(f(X) + \epsilon - \hat{f}(X))^2\right] \\
&= E\left[(f(X) - \hat{f}(X) + \epsilon)^2\right] \\
&= E\left[(f(X) - \hat{f}(X))^2 + 2\epsilon(f(X) - \hat{f}(X)) + \epsilon^2\right] \\
&= E\left[(f(X) - \hat{f}(X))^2\right] + 2E\left[\epsilon(f(X) - \hat{f}(X))\right] + E\left[\epsilon^2\right]
\end{aligned}$$

Simplify those last two terms.

- **The Cross Term**: Because the noise ϵ is random and independent of our model, and its average is 0 ($E[\epsilon] = 0$), the middle term disappears:

$$2E\left[\epsilon(f(X) - \hat{f}(X))\right] = 2E[\epsilon]E[f(X) - \hat{f}(X)] = 0$$

- **The Noise Term**: The definition of variance is $\text{Var}(\epsilon) = E[\epsilon^2] - (E[\epsilon])^2$. Since $E[\epsilon] = 0$, we get $E[\epsilon^2] = \text{Var}(\epsilon) = \sigma^2$. This is the **irreducible error**.

After simplifying, our MSE equation looks like this:

$$\text{MSE} = E\left[(f(X) - \hat{f}(X))^2\right] + \sigma^2$$

Step 2: Decompose the Model Error: The remaining term, $E\left[(f(X) - \hat{f}(X))^2\right]$, represents the error that comes purely from our model.

To break it down, we use a common trick: adding and subtracting the *average prediction* of our model, $E[\hat{f}(X)]$.

$$\begin{aligned} E\left[(f(X) - \hat{f}(X))^2\right] &= E\left[(f(X) - E[\hat{f}(X)] + E[\hat{f}(X)] - \hat{f}(X))^2\right] \\ &= E\left[(f(X) - E[\hat{f}(X)])^2 + (\hat{f}(X) - E[\hat{f}(X)])^2 - 2(...)\right] \end{aligned}$$

When we expand this, the cross-product term becomes zero for the same reason as before (the expectation of a deviation from its mean is zero). We are left with two meaningful parts:

1. $E\left[(f(X) - E[\hat{f}(X)])^2\right] = (f(X) - E[\hat{f}(X)])^2 = (\text{Bias})^2$. This is the squared **Bias**: the difference between the true function and the *average prediction* of our model. It captures how systematically wrong our model is.
2. $E\left[(\hat{f}(X) - E[\hat{f}(X)])^2\right] = \text{Var}(\hat{f}(X))$. This is the **Variance**: how much our model's prediction $\hat{f}(X)$ fluctuates around its average for different training sets. It captures the model's sensitivity to the specific data it was trained on.

Step 3: By substituting these components back into our MSE equation, we get the final decomposition:

$$\text{MSE} = \underbrace{(E[\hat{f}(X)] - f(X))^2}_{\text{Bias}^2} + \underbrace{E\left[(\hat{f}(X) - E[\hat{f}(X)])^2\right]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible Error}}$$

Application (Part A): Basics (65 points)

Dataset: `Boston Housing`, saved as `assign1_regData.csv`.

Harrison and Rubinfeld (1978)[1] studied families' willingness to pay for clean air using Boston area housing market data in 1978. The cleaned data consists of 394 rows with **complete** data on census tracts in Boston, with variables collected from a mixture of surveys, administrative records, and other research papers. The variables are described below.

- `CRIM`: per-capita crime rate by town (units and crimes included not specified)
- `ZN`: percentage of residential land zoned for lots over 25,000 sq.ft.
- `INDUS`: proportion of non-retail business acres per town (nonretail businesses include industrial and commercial businesses)
- `CHAS`: Charles River dummy variable (= 1 if tract bounds river; 0 otherwise)
- `NOX`: nitric oxides concentration (parts per 10 million)
- `RM`: average number of rooms per dwelling
- `AGE`: percentage of owner-occupied units built prior to 1940
- `DIS`: weighted log distances to five Boston employment centers
- `RAD`: index of accessibility to radial highways, where a higher number indicates higher accessibility

[1] David Harrison, Jr and Daniel L Rubinfeld. Hedonic housing prices and the demand for clean air. Journal of Environmental Economics and Management, 5(1):81–102, 1978.

- `TAX`: full-value property-tax rate per $10,000
- `PTRATIO`: pupil-teacher ratio by school district
- `B`: $1000(B_k - 0.63)^2$ where B_k is the proportion of Black people by town
- `LSTAT`: % lower status of the population, defined as half of the sum of proportion of adults without some high school education and proportion of male workers classified as laborers
- `MEDV`: Median value of owner-occupied homes, in $1000's

We will load the dataset and set the outcome variable to be `MED`.

```r
# Load required libraries and dataset
pacman::p_load(tidyverse, caret, boot, glmnet, splines)
data <- read_csv("assign1_regData.csv") |>
    drop_na()
data$CHAS <- as.factor(data$CHAS)
```

A1. Data Exploration & Baseline Model (5 + 5 + 5 = 15 points)

1. Use the `summary` function to provide descriptive statistics for the dataset. Randomly split your data into a training set (80%) and a test set (20%). Make sure that the distribution of the outcome is similar in both sets (please read the documentation for `caret::createDataPartition` to know how). Use a fixed seed (use `seed = 42`) for reproducibility.

```r
summary(data)
```

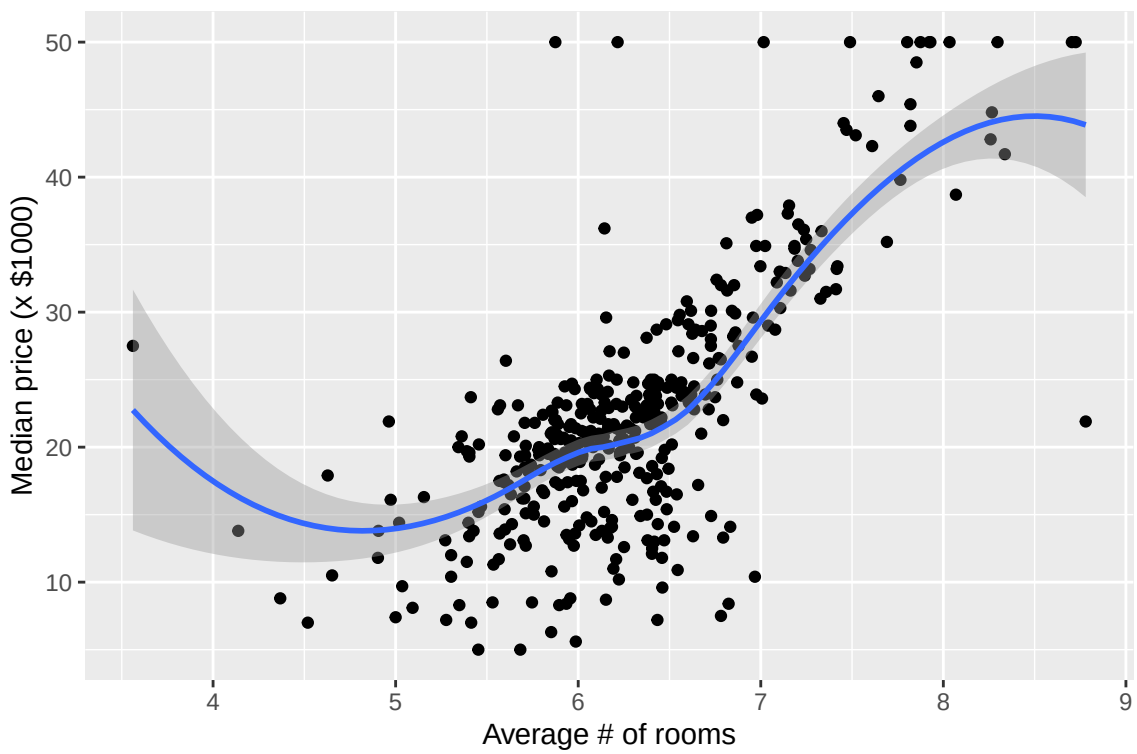
```
##       CRIM                ZN             INDUS       CHAS         NOX
##  Min.   : 0.00632   Min.   :  0.00   Min.   : 0.46   0:367   Min.   :0.3890
##  1st Qu.: 0.08196   1st Qu.:  0.00   1st Qu.: 5.13   1: 27   1st Qu.:0.4530
##  Median : 0.26888   Median :  0.00   Median : 8.56           Median :0.5380
##  Mean   : 3.69014   Mean   : 11.46   Mean   :11.00           Mean   :0.5532
##  3rd Qu.: 3.43597   3rd Qu.: 12.50   3rd Qu.:18.10           3rd Qu.:0.6240
##  Max.   :88.97620   Max.   :100.00   Max.   :27.74           Max.   :0.8710
##        RM              AGE              DIS              RAD
##  Min.   :3.561   Min.   :  2.90   Min.   : 1.130   Min.   : 1.000
##  1st Qu.:5.879   1st Qu.: 45.48   1st Qu.: 2.110   1st Qu.: 4.000
##  Median :6.202   Median : 77.70   Median : 3.199   Median : 5.000
##  Mean   :6.280   Mean   : 68.93   Mean   : 3.805   Mean   : 9.404
##  3rd Qu.:6.606   3rd Qu.: 94.25   3rd Qu.: 5.117   3rd Qu.:24.000
##  Max.   :8.780   Max.   :100.00   Max.   :12.127   Max.   :24.000
##       TAX           PTRATIO            B              LSTAT
##  Min.   :187.0   Min.   :12.60   Min.   :  2.6   Min.   : 1.730
##  1st Qu.:280.2   1st Qu.:17.40   1st Qu.:376.7   1st Qu.: 7.125
##  Median :330.0   Median :19.10   Median :392.2   Median :11.300
##  Mean   :406.4   Mean   :18.54   Mean   :358.5   Mean   :12.769
##  3rd Qu.:666.0   3rd Qu.:20.20   3rd Qu.:396.9   3rd Qu.:17.117
##  Max.   :711.0   Max.   :22.00   Max.   :396.9   Max.   :37.970
##       MEDV
##  Min.   : 5.00
##  1st Qu.:16.80
##  Median :21.05
##  Mean   :22.36
##  3rd Qu.:25.00
##  Max.   :50.00
```

```r
set.seed(42)
# we'll split based on the MEDV column to maintain its
# distribution in both sets.
```

```r
trainIndex <- createDataPartition(data$MEDV, p = 0.8, list = FALSE,
    times = 1)
# Create the training set
train_set <- data[trainIndex, ]
# Create the test set
test_set <- data[-trainIndex, ]
# You can verify the dimensions of your new datasets
# cat('Training set dimensions:', paste(dim(train_set),
# collapse = '*')) cat('Test set dimensions:',
# paste(dim(test_set), collapse = '*'))
```

2. Perform a brief exploratory data analysis: comment on how the outcome varies with the `RM` predictor - is the behavior expected?

```r
data.frame(x = data$RM, y = data$MEDV) |>
    ggplot(aes(x = x, y = y)) + geom_point() + geom_smooth() +
    labs(x = "Average # of rooms", y = "Median price (x $1000)")
```



Yes, the behavior is expected - houses with more rooms are going to be bigger and likely be more expensive on average. There are some y-outliers for very small or very large x but we see an overall increasing trend of higher cost for more rooms on average.

3. Fit a multiple linear regression model using all predictors on the training data. Report the test set Root Mean Squared Error (RMSE).

```r
lm_model <- lm(MEDV ~ ., data = train_set)
predictions <- predict(lm_model, newdata = test_set)
rmse <- sqrt(mean((predictions - test_set$MEDV)^2))
print(rmse)
```

```
## [1] 5.651607
```

A2. Resampling and Model Assessment (5 + 8 + 2 = 15 points)

1. Using the training data, perform 10-fold cross-validation on the full multiple linear regression model from A1. What is the average cross-validated RMSE?

```r
set.seed(42)
cv_control <- trainControl(method = "cv", number = 10)
full_lm_model_cv <- train(MEDV ~ .,
                          data = train_set,
                          method = "lm", # "lm" specifies a linear model
                          trControl = cv_control,
                          metric = "RMSE")

# use this for all details related to your k-fold CV routine.
# print(full_lm_model_cv)

# You can also access the average RMSE directly from the results
avg_cv_rmse <- full_lm_model_cv$results$RMSE
print(paste("The average 10-fold cross-validated RMSE is:", round(avg_cv_rmse, 4)))
```

```
## [1] "The average 10-fold cross-validated RMSE is: 4.2739"
```

2. Use the bootstrap method (with $B = 1000$ resamples, set `seed = 42`) on the training data to estimate the 95% confidence interval (CI) for the `RM` coefficient. Compare the bootstrap-based 95% CI with the 95% CI using the standard formula from our linear regression model.

```r
### running the bootstrap
get_coefficient <- function(data, indices) {
    resampled_data <- data[indices, ]
    model <- lm(MEDV ~ ., data = resampled_data)
    return(coef(model)["RM"])
}

set.seed(42)
bootstrapped_coef <- replicate(1000, get_coefficient(data = train_set,
    indices = sample(1:nrow(train_set), replace = TRUE)))
```

```r
# Print the bootstrap confidence interval
print(quantile(bootstrapped_coef, probs = c(0.025, 0.975)))
```

```
##     2.5%    97.5%
## 2.211581 5.885868
```

```r
# Print the OLS theory-based CI
confint(lm_model, parm = "RM")
```

```
##       2.5 %   97.5 %
## RM 2.756063 4.746361
```

3. Interpretation: *The standard 95% CI from a linear regression is parametric, meaning it relies on a strict set of mathematical assumptions to be accurate. The bootstrap CI is non-parametric; it builds an empirical estimate of the sampling distribution by resampling our data, making it robust to violations of those assumptions. Comparing CIs suggests that the standard regression model is underestimating the true variability of our parameter estimate. The bootstrap method has detected instability or patterns in the data that the standard method overlooks. We should run diagnostic plots on our linear regression*

model to confirm which assumptions are being violated. By means of model diagnostics, a Q-Q plot of the residuals will check for normality and a residuals vs. fitted values plot will check for heteroscedasticity and outliers.

A3. Feature Selection & Regularization (8 + 4 + 4 + 4 = 20 points)

1. Stepwise Selection: On the training data, perform forward stepwise selection to find the best model according to BIC. List the variables selected by the procedure

```
step_fwd_bic <- step(object = lm(MEDV ~ 1, data = train_set),
    scope = list(lower = formula(lm(MEDV ~ 1, data = train_set)),
        upper = formula(lm(MEDV ~ ., data = train_set))), direction = "forward",
    k = log(nrow(train_set)), trace = 0  ## making the algorithm run quietly
)
paste0("Selected variables: ", paste0(names(coefficients(step_fwd_bic))[-1],
    collapse = ",  "))
```

```
## [1] "Selected variables: LSTAT,  RM,  PTRATIO,  B,  DIS,  NOX,  ZN,  CRIM"
```

2. Ridge Regression: Fit a ridge regression model, using 10-fold CV to determine the optimal lambda for the training set. Report this lambda and the test set RMSE.

```
set.seed(42)
train_control <- trainControl(method = "CV", number = 10)
ridge_model <- train(MEDV ~ ., data = train_set, method = "glmnet",
    trControl = train_control, tuneLength = ncol(train_set) -
        1, tuneGrid = expand.grid(alpha = 0, lambda = 10^seq(-3,
        1, length = 1000)), preProcess = c("center", "scale"))
pred_ridge <- predict(ridge_model, newdata = test_set)
rmse_ridge <- sqrt(mean((pred_ridge - test_set$MEDV)^2))
```

```
paste0("Optimal lambda (for ridge): ", round(ridge_model$bestTune$lambda,
    2), " and test set RMSE: ", round(rmse_ridge, 2))
```

```
## [1] "Optimal lambda (for ridge): 0.65 and test set RMSE: 5.69"
```

3. Lasso Regression: Fit a lasso regression model. Use 10-fold CV to determine the optimal lambda. Report optimal lambda, list variables shrunk to zero, and calculate test set RMSE. Finally, plot coefficient paths for varying lambda.

```
set.seed(42)
train_control <- trainControl(method = "CV", number = 10)
lasso_model <- train(MEDV ~ ., data = train_set, method = "glmnet",
    trControl = train_control, tuneLength = ncol(train_set) -
        1, tuneGrid = expand.grid(alpha = 1, lambda = 10^seq(-3,
        1, length = 1000)), preProcess = c("center", "scale"))
pred_lasso <- predict(lasso_model, newdata = test_set)
rmse_lasso <- sqrt(mean((pred_lasso - test_set$MEDV)^2))
```

```
paste0("Optimal lambda (for lasso): ", round(lasso_model$bestTune$lambda,
    2), " and test set RMSE: ", round(rmse_lasso, 2))
```

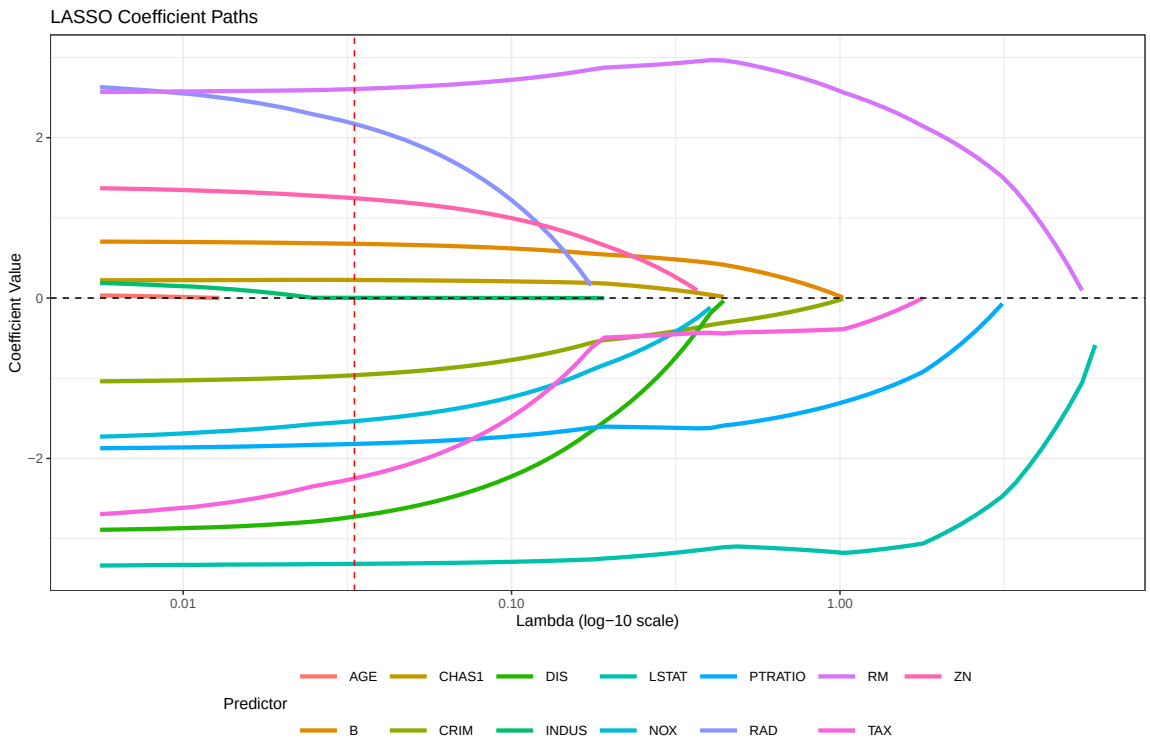
```
## [1] "Optimal lambda (for lasso): 0.03 and test set RMSE: 5.65"
```

```
shrunken_vars <- data.frame(as.matrix(predict(lasso_model$finalModel,
    type = "coefficients", s = lasso_model$bestTune$lambda))) |>
    filter(s1 == 0) |>
    rownames()
```

```r
paste0("Variables Shrunk to Zero: ", paste0(shrunken_vars, collapse = ", "))
```

```
## [1] "Variables Shrunk to Zero: INDUS, AGE"
```

```r
coef_paths <- broom::tidy(lasso_model$finalModel) |>
  filter(term != "(Intercept)")
ggplot(coef_paths, aes(x = lambda, y = estimate,
                       group = term, color = term)) +
  geom_line(linewidth = 0.75) +
  scale_x_log10() + # Lambda is best viewed on a log scale
  geom_vline(xintercept = lasso_model$bestTune$lambda,
             linetype = "dashed", color = "red") +
  geom_hline(yintercept = 0, linetype = "dashed",
             color = "black") +
  labs(
    title = "LASSO Coefficient Paths",
    x = "Lambda (log-10 scale)",
    y = "Coefficient Value",
    color = "Predictor"
  ) +
  theme_bw(base_size = 6) +
  theme(legend.position = "bottom") +
  guides(colour = guide_legend(nrow = 2))
```



4. Partial Least Squares (PLS): Fit a PLS model. Use 10-fold CV to select the optimal number of components. Report the test set RMSE.

```r
set.seed(42)
train_control <- trainControl(method = "CV", number = 10)
pls_model <- train(MEDV ~ ., data = train_set, method = "pls",
```

```
    trControl = train_control, tuneLength = ncol(train_set) -
        1, preProcess = c("center", "scale"))
pred_pls <- predict(pls_model, newdata = test_set)
rmse_pls <- sqrt(mean((pred_pls - test_set$MEDV)^2))
```

```
paste0("Optimal # of components: ", pls_model$bestTune, " and test set RMSE: ",
    round(rmse_pls, 2))
```

```
## [1] "Optimal # of components: 7 and test set RMSE: 5.62"
```

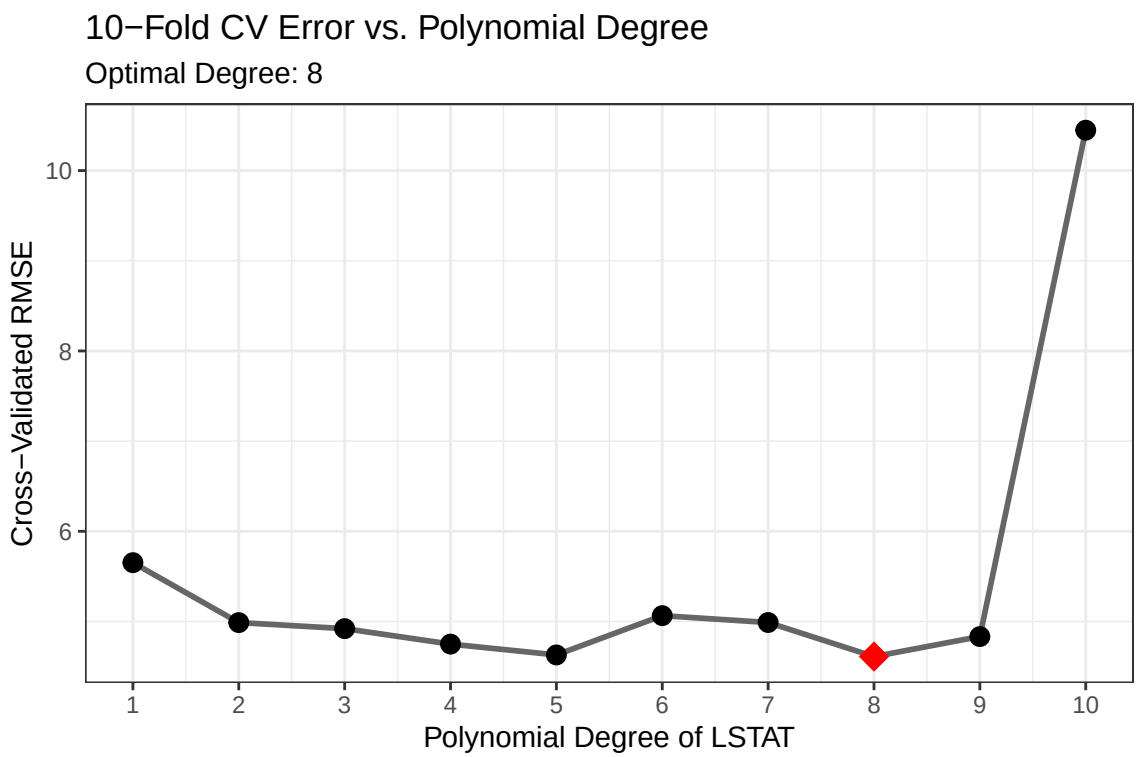
A4. Non-Linear Modeling (5 + 5 + 5 = 15 points)

1. Polynomial Regression: Consider a polynomial regression with `LSTAT` as the only predictor. Use 10-fold CV to select the optimal polynomial degree (1 to 10). Plot how CV errors vary with polynomial degree and the optimal degree. Report test set RMSE for the optimal model.

```
set.seed(42)
cv_results <- tibble(degree = 1:10, cv_rmse = NA)
for (d in 1:10) {
    formula <- as.formula(paste("MEDV ~ poly(LSTAT,", d, ")"))
    poly_model <- train(form = formula, data = train_set, method = "lm",
        preProcess = c("center", "scale"), trControl = train_control)
    cv_results$cv_rmse[d] <- poly_model$results$RMSE
}

best_degree <- cv_results$degree[which.min(cv_results$cv_rmse)]
final_formula <- as.formula(paste("MEDV ~ poly(LSTAT,", best_degree,
    ")"))
final_model <- train(form = final_formula, data = train_set,
    method = "lm")
predictions <- predict(final_model, newdata = test_set)
test_rmse <- RMSE(predictions, test_set$MEDV)
```

```
ggplot(cv_results, aes(x = degree, y = cv_rmse)) + geom_line(color = "gray40",
    size = 1) + geom_point(color = "black", size = 3) + geom_point(data = filter(cv_results,
    degree == best_degree), aes(x = degree, y = cv_rmse), color = "red",
    size = 5, shape = 18) + scale_x_continuous(breaks = 1:10) +
    labs(title = "10-Fold CV Error vs. Polynomial Degree", subtitle = paste("Optimal Degree:",
        best_degree), x = "Polynomial Degree of LSTAT", y = "Cross-Validated RMSE") +
    theme_bw()
```




```
paste0("Test Set RMSE for the optimal model (degree=", best_degree,
    "): ", round(test_rmse, 2))
```

```
## [1] "Test Set RMSE for the optimal model (degree=8): 6.91"
```

2. Splines: Fit a natural spline (remember, natural spline = cubic spline + linearity in tails) with `RM` as the predictor. Report test set RMSE.

```
set.seed(42)
cv_results <- tibble(degree = 1:5, cv_rmse = NA)
train_control <- trainControl(method = "CV", number = 10)
for (d in 1:5) {
    formula <- as.formula(paste("MEDV ~ ns(RM,", d, ")"))
    set.seed(42)
    spline_model <- train(form = formula, data = train_set, method = "lm",
        preProcess = c("center", "scale"), trControl = train_control)
    cv_results$cv_rmse[d] <- spline_model$results$RMSE
}
best_degree <- cv_results$degree[which.min(cv_results$cv_rmse)]
final_formula <- as.formula(paste("MEDV ~ ns(RM,", best_degree,
    ")"))
final_model <- train(form = final_formula, data = train_set,
    method = "lm")
predictions <- predict(final_model, newdata = test_set)
test_rmse <- RMSE(predictions, test_set$MEDV)
```

```
paste0("Test Set RMSE for the optimal model (df=", best_degree,
    "): ", round(test_rmse, 2))
```

```
## [1] "Test Set RMSE for the optimal model (df=5): 6.69"
```

3. Generalized Additive Model (GAM): Fit three GAMs, each with with natural splines for (a) `LSTAT` alone, (b) `RM` alone, and (c) both `LSTAT` and `RM`. Compare the three models and for the optimal one, report test set RMSE.

```
library(mgcv)
test_x <- model.matrix(MEDV ~ ., data = test_set)[, -1]
test_y <- test_set$MEDV
model_1 <- gam(MEDV ~ ns(LSTAT), data = train_set)
model_2 <- gam(MEDV ~ ns(RM), data = train_set)
model_3 <- gam(MEDV ~ ns(LSTAT) + ns(RM), data = train_set)
anova(model_1, model_2, model_3)
```

```
## Analysis of Deviance Table
##
## Model 1: MEDV ~ ns(LSTAT)
## Model 2: MEDV ~ ns(RM)
## Model 3: MEDV ~ ns(LSTAT) + ns(RM)
##   Resid. Df Resid. Dev Df Deviance      F    Pr(>F)
## 1       315      10174
## 2       315      11605  0  -1431.3
## 3       314       7929  1   3675.8 145.57 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
gam.pred <- predict(model_3, data.frame(test_x))
sqrt(mean((gam.pred - test_y)^2))
```

```
## [1] 6.569848
```

Application (Part B): Advanced (25 points)

The `B` variable in this data set is defined as $1000(B_k - 0.63)^2$ where B_k is the proportion of Black people by town. Due to the squared term, this variable treats towns with very low proportions of Black residents and towns with very high proportions similarly, which is a problematic construction. This exercise will investigate how models trained on demographically distinct subgroups can exhibit significant performance bias.

You will train two separate models on two different subgroups of the data and then evaluate their performance on both their own and each other's test sets.

1. Data Partitioning (2 points): First, partition the entire data into two groups based on the median of the `B` variable.
   - Group A: All rows where the `B` value is less than or equal to the median of `B`.
   - Group B: All rows where the `B` value is greater than the median of `B`.

```
library(gam)
group_a <- data |>
    filter(B <= median(data$B))
group_b <- data |>
    filter(B > median(data$B))
```

2. Stratified Splitting (8 points): For each group, perform an 80/20 train-test split (remember to set `seed = 42`). Crucially, this split must be stratified by the outcome variable, `MEDV`, to ensure that the distribution of housing prices is similar in both the training and testing sets for each group. This will result in four datasets: `Train A`, `Test A`, `Train B`, and `Test B`.

```r
set.seed(42)
# Stratified split for Group A
train_index_a <- createDataPartition(group_a$MEDV, p = 0.8, list = FALSE)
train_a <- group_a[train_index_a, ]
test_a <- group_a[-train_index_a, ]
# Stratified split for Group B
train_index_b <- createDataPartition(group_b$MEDV, p = 0.8, list = FALSE)
train_b <- group_b[train_index_b, ]
test_b <- group_b[-train_index_b, ]
cat("Group A - Train:", nrow(train_a), "| Test:", nrow(test_a),
    "\n")
```

```
## Group A - Train: 159 | Test: 38
```

```r
cat("Group B - Train:", nrow(train_b), "| Test:", nrow(test_b),
    "\n\n")
```

```
## Group B - Train: 159 | Test: 38
```

3. Model Training (10 points): Train two distinct local regression models using the training data from your subgroups:
   - `Model A`: Fit a LOESS model with `span = 0.15` using `Train A` using `RM` and `LSTAT` as predictors.
   - `Model B`: Fit a LOESS model with `span = 0.15` using `Train B` using `RM` and `LSTAT` as predictors.

```r
# Fit group-level models
model_a <- gam(MEDV ~ lo(LSTAT, span = 0.15) + lo(RM, span = 0.15),
    data = train_a)
model_b <- gam(MEDV ~ lo(LSTAT, span = 0.15) + lo(RM, span = 0.15),
    data = train_b)
```

4. Performance Evaluation and Interpretation (5 points): Calculate the Root Mean Squared Error (RMSE) for the following four scenarios. This will involve evaluating each model on both its "in-group" test set and the "cross-group" test set.
   - In-Group A: The error of `Model A` when predicting `Test A`.
   - In-Group B: The error of `Model B` when predicting `Test B`.
   - Cross-Group $A \to B$: The error of `Model A` when predicting `Test B`.
   - Cross-Group $B \to A$: The error of `Model B` when predicting `Test A`.

Present your four RMSE values in a table. Compare the in-group versus cross-group error rates. What do these results tell you about how the B variable influences model performance? Explain what this implies about the dangers of training and deploying models on demographically specific or unrepresentative data.

```r
# Make predictions for all four scenarios
pred_a_on_a <- predict(model_a, newdata = test_a)
pred_b_on_b <- predict(model_b, newdata = test_b)
pred_a_on_b <- predict(model_a, newdata = test_b)
pred_b_on_a <- predict(model_b, newdata = test_a)
# Calculate RMSE for each scenario
rmse_a_on_a <- RMSE(pred_a_on_a, test_a$MEDV)
rmse_b_on_b <- RMSE(pred_b_on_b, test_b$MEDV)
rmse_a_on_b <- RMSE(pred_a_on_b, test_b$MEDV)
rmse_b_on_a <- RMSE(pred_b_on_a, test_a$MEDV)
# Create a results table
results_table <- tibble(Model_Used = c("Model A", "Model B",
    "Model A", "Model B"), TestData_From = c("Test A", "Test B",
```

```r
    "Test B", "Test A"), Scenario = c("In-Group A", "In-Group B",
    "Cross-Group A->B", "Cross-Group B->A"), RMSE = c(rmse_a_on_a,
    rmse_b_on_b, rmse_a_on_b, rmse_b_on_a))
print(results_table)
```

```
## # A tibble: 4 x 4
##   Model_Used TestData_From Scenario          RMSE
##   <chr>      <chr>         <chr>            <dbl>
## 1 Model A    Test A        In-Group A        5.42
## 2 Model B    Test B        In-Group B        2.77
## 3 Model A    Test B        Cross-Group A->B  3.91
## 4 Model B    Test A        Cross-Group B->A  4.56
```

Bonus (unscored, will still provide feedback if you answer)

1. LOO-CV: Prove formula (5.2) from ISLR, Section 5.1.2, page 202.
2. Basis of 0.632 bootstrap: Problem 5.4.2 from ISLR, page 219.
3. Continuity and smoothness of cubic splines: Problem 7.9.1 from ISLR, page 321.